CHAPTER 7

CONTROL STATEMENTS

CHAPTER ESSENCE AT A GLANCE:
Master the algorithmic flow control mechanisms of programming governing conditional execution branching, entry-controlled and exit-controlled loops, and jump disruptions.

4 CORE PRINCIPLES TO UNDERSTAND FIRST

• **Branching and Selection Logic:** Selection statements evaluate conditional boolean expressions to route program execution down mutually exclusive logical pathways.

• **Switch Case Jump Tables:** The switch construct performs multi-way branching against integral or character constant expressions using break statements to prevent fall-through.

• **Entry versus Exit Loop Protocols:** Entry-controlled loops verify conditions before entering the loop body and may execute zero times, whereas exit-controlled do-while loops guarantee at least one execution.

• **Loop Control Interruption:** The break statement terminates the innermost enclosing loop immediately, while continue skips the remainder of the current iteration to evaluate the next cycle.

ESSENTIAL ARCHITECTURE & CORE SYNTAX CARDS

$$\text{for}(\text{init; condition; update}) \{ \text{Loop Body Statements} \}$$

Entry-Controlled For Loop Syntax

$$\text{do} \{ \text{Body} \} \text{while}(\text{condition}); \quad (\text{Minimum 1 Execution})$$

Exit-Controlled Do-While Loop

$$\text{switch}(\text{expr}) \rightarrow \text{case } C_1 : \text{break}; \quad \dots \quad \text{default} :$$

Multi-Way Branching Jump Table

$$\text{break} \rightarrow \text{Exit Loop} \quad | \quad \text{continue} \rightarrow \text{Next Iteration}$$

Loop Flow Interruption Keywords

CRUCIAL EXAM PRECAUTIONS & VALUATION RUBRIC SECRETS

• **Alert:** A do-while loop MUST conclude with a terminating semicolon directly after the closing while condition: while(condition);.

• **Alert:** Omitting the break; statement inside a switch block leads to unintended fall-through, executing all subsequent cases sequentially.

• **Alert:** Placing an accidental semicolon directly after a for loop header or if condition creates an empty statement, causing severe logical errors.

HIGH-YIELD CENTUM QUESTIONS & STEP-BY-STEP EXPLANATIONS

1. Distinguish between entry-controlled loop and exit-controlled loop with one example of each. [Score: 2 Marks]

Answer & Key Valuation Steps:

• **Entry-Controlled Loop:** The test condition is checked before executing the loop body; if the condition is false initially, the body is never executed (e.g. for loop, while loop). **[1 Mark]**

• **Exit-Controlled Loop:** The test condition is checked after executing the loop body; the body is guaranteed to execute at least once regardless of condition (e.g. do-while loop). **[1 Mark]**

2. What is the role of the default label in a switch statement? Is it mandatory? [Score: 2 Marks]

Answer & Key Valuation Steps:

• **Role:** The default block executes when the value of the switch expression fails to match any of the enumerated case constant values. **[1 Mark]**

• **Optional Nature:** The default label is completely optional; if omitted and no case matches, the entire switch construct terminates with zero action taken. **[1 Mark]**

3. Compare if-else ladder and switch statement on three key programming criteria. [Score: 3 Marks]

Answer & Key Valuation Steps:

• **Expression Evaluation:** if-else evaluates expressions based on ranges, logical conditions (&&, ||), and floating-point comparisons; switch tests only for equality against integer or character constants. **[1 Mark]**

• **Execution Mechanism:** if-else evaluates conditions sequentially from top to bottom; switch generates jump tables enabling direct constant jumps. **[1 Mark]**

• **Readability:** switch provides cleaner, more readable structure when handling numerous discrete constant options compared to complex nested if-else ladders. **[1 Mark]**

4. Differentiate between break and continue jump statements in C++ loops with syntax examples. [Score: 3 Marks]

Answer & Key Valuation Steps:

• **break Statement:** Causes immediate premature termination of the innermost enclosing loop or switch construct, transferring control to the statement following the loop. **[1.5 Marks]**

• **continue Statement:** Skips the remaining statements within the current loop iteration and immediately triggers the next iteration (evaluating increment and condition). **[1.5 Marks]**

5. Convert the following while loop into an equivalent for loop: int i = 1, sum = 0; while (i <= 50) { sum += i; i += 2; } [Score: 3 Marks]

Answer & Key Valuation Steps:

• **Loop Anatomy Analysis:** Initialization is i = 1, test condition is i <= 50, update expression is i += 2, and body is sum += i. **[1 Mark]**

• **Equivalent For Loop Structure:** int sum = 0;nfor (int i = 1; i ≤ 50; i += 2) {n sum += i;n} **[1.5 Marks]**

• **Verification:** Both constructs execute identical 25 iterations accumulating odd integers up to 50. Boxed Result: Clean syntactic transformation. **[0.5 Mark]**

6. Write a complete C++ program to check whether a given positive integer is a Prime Number. [Score: 4 Marks]

Answer & Key Valuation Steps:

• **Program Setup & Input:** #include <iostream>vsing namespace std;nint main() { int num; bool isPrime = true; cout << 'Enter number: '; cin >> num; **[1 Mark]**

• **Corner Case & Trial Division:** if (num <= 1) isPrime = false; else { for (int i = 2; i * i <= num; i++) { if (num % i == 0) { isPrime = false; break; } } } **[1.5 Marks]**

• **Output & Boxed Result:** if (isPrime) cout << num << ' is Prime'; else cout << num << ' is Not Prime'; return 0; } Boxed Result: Evaluates primality in O(sqrt(N)) time. **[1.5 Marks]**

7. Write a C++ program to reverse a given integer number and check whether it is a Palindrome. [Score: 3 Marks]

Answer & Key Valuation Steps:

• **Initialization & Extraction:** int n, num, rev = 0, rem; cin >> n; num = n; while (n > 0) { rem = n % 10; rev = rev * 10 + rem; n /= 10; } **[1.5 Marks]**

• **Comparison & Result:** if (num == rev) cout << 'Palindrome'; else cout << 'Not Palindrome'; **[1 Mark]**

• **Boxed Result:** Integer reversed via decimal digit extraction and verified. **[0.5 Mark]**